UNIVERSIDADE DA CORUÑA

Práctica 2

Clasificación de Imágenes Portuarias

Visión por Computador Aplicada

Autor: Miguel Baños Baladrón
Autor: Fiz Garrido Escudero
Fecha: 19 de abril de 2026

1. Introducción

En el contexto de una plataforma *Smartports*, el módulo de videovigilancia detecta automáticamente eventos de atraque y desatraque analizando el flujo de vídeo de cámaras CCTV instaladas en el puerto. Esta práctica aborda dos tareas de clasificación binaria sobre imágenes estáticas:

- **Ship / No-Ship**: determinar si hay algún barco visible en la escena.
- **Docked / Undocked**: dado que hay un barco, determinar si está atracado.

Para cada tarea se entrenan y evalúan cuatro configuraciones: una red convolucional base (*BaseCNN*) y un modelo preentrenado (*ResNet18*), cada uno con y sin *data augmentation*. Todos los experimentos se ejecutan además en dos variantes del dataset: con y sin el *overlay* de la cámara CCTV.

2. Análisis del dataset

El dataset consta de **294 imágenes** capturadas por distintas cámaras en entornos portuarios, con variabilidad significativa de ángulo, escala, hora del día y condiciones de iluminación. La distribución de clases se recoge en la Tabla 1.

Cuadro 1: Distribución de clases en el dataset completo.

Clase	Imágenes	Proporción
Ship	184	62,6 %
No-Ship	110	37,4 %
Docked	92	50,0 % de Ship
Undocked	92	50,0 % de Ship

Ship/No-Ship presenta un desbalance moderado (≈63/37), que se compensa durante el entrenamiento mediante `pos_weight` en la función de pérdida. Docked/Undocked está perfectamente balanceado entre las imágenes con barco.

Variabilidad visual

Más allá del balance numérico, conviene inspeccionar visualmente las imágenes para detectar tres factores que condicionan la dificultad del problema: **casos ambiguos** (barcos lejanos o muelles vacíos con apariencia similar a escenas con barco), **variabilidad intraclase** (imágenes de la misma categoría con aspecto muy distinto) y **patrones de fondo** que puedan inducir correlaciones espurias. La Figura 1 recoge muestras representativas de cada clase, seleccionadas de forma uniforme por índice para reflejar la variedad real del dataset y no casos extremos.

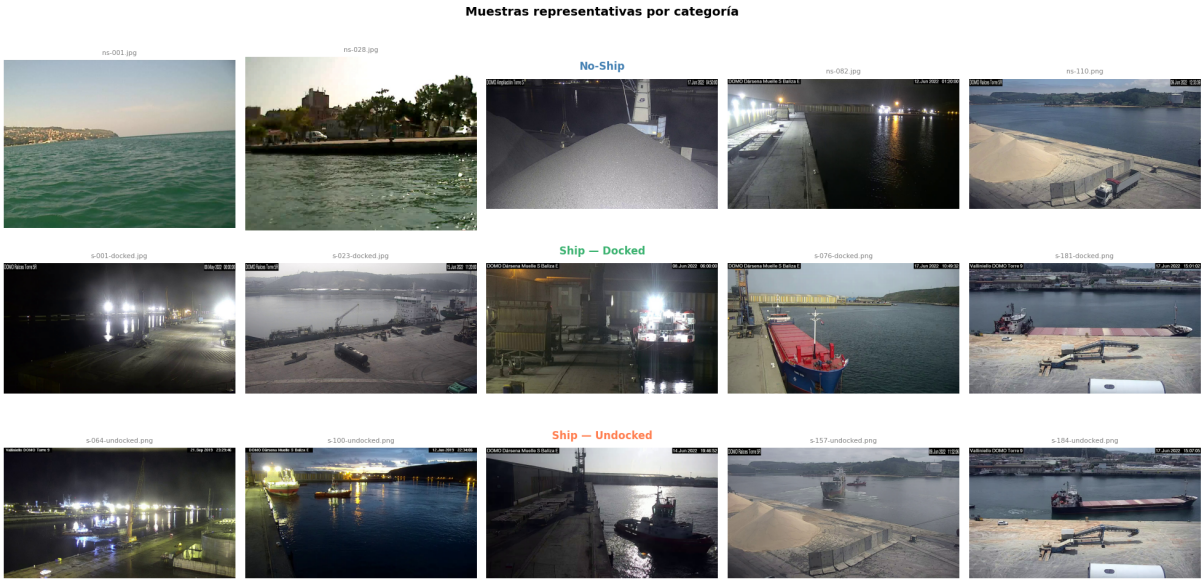


Figura 1: Muestras representativas por clase.

Distribución visual — UMAP

Para explorar la estructura del espacio de características se aplica UMAP sobre embeddings ResNet18 de las 294 imágenes. La Figura 2 muestra la proyección 2D y 3D coloreada por etiquetas

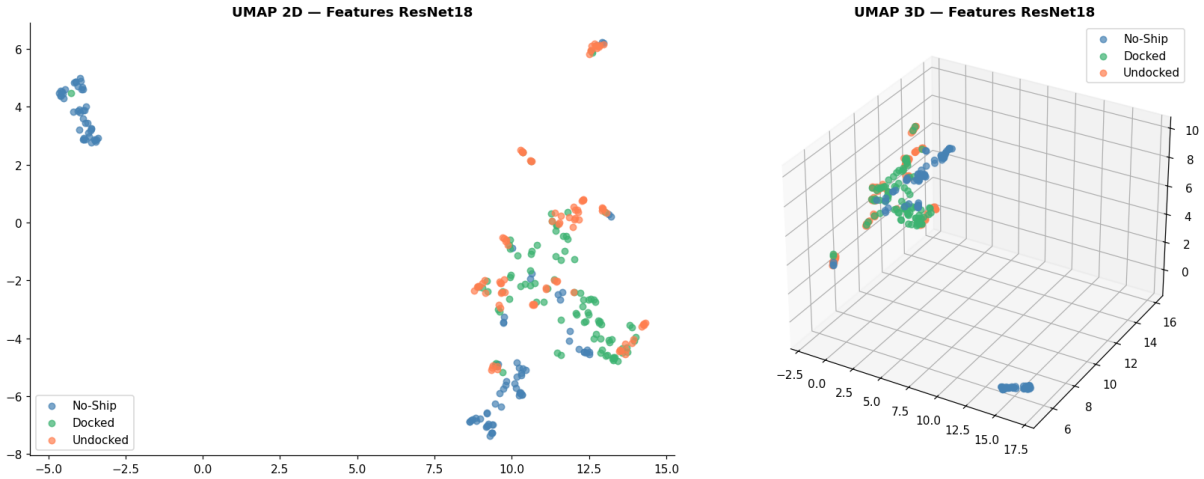


Figura 2: Proyección UMAP del dataset.

En la proyección se aprecian dos regiones de imágenes etiquetadas como No-Ship, relativamente separadas. Lo más probable, es que esas imágenes No-Ship separadas del resto sean las 44 imágenes que no muestran overlay, lo que reafirma nuestra creencia de que los modelos pueden llegar a aprender correlaciones espurias.

La separación es nula en Docked/Undocked, lo que deja entrever la mayor dificultad de esta tarea: ambas clases comparten el mismo tipo de escena (barco presente) y la diferencia es puramente geométrica.

Condiciones de iluminación y contraste

La Figura 3 muestra la distribución de brillo medio y contraste por imagen, coloreada por clases. El dataset cubre un rango amplio: desde imágenes nocturnas de bajo brillo y contraste hasta escenas diurnas despejadas de alta luminosidad.

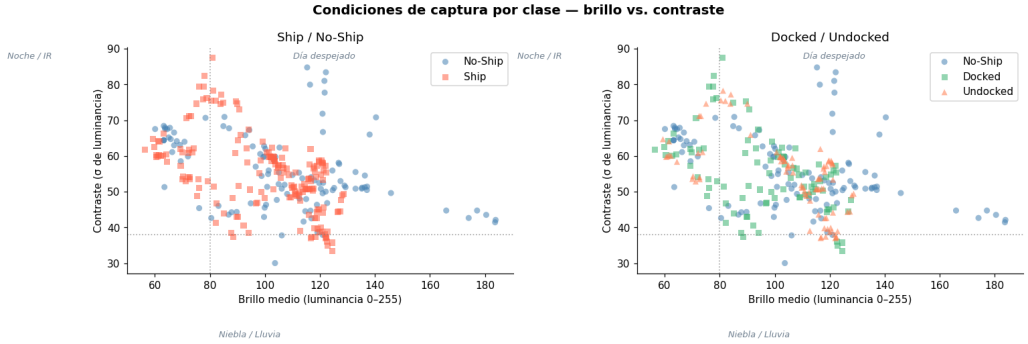


Figura 3: Brillo medio vs. contraste..

Podemos ver diferencias entre las clases Ship/No-ship, que pueden derivar en correlaciones espurias, como que no encontramos en el dataset imágenes que tengan barcos presentes y sean a la vez muy brillantes.

La variabilidad observada justifica el uso de `ColorJitter` y `GaussianBlur` en el *data augmentation* con parámetros derivados directamente de estas estadísticas (σ/μ de brillo y contraste para `ColorJitter`, percentil 10 de contraste para $\sigma_{\text{máx}}$ de `GaussianBlur`), en lugar de valores arbitrarios.

Sesgo estructural del overlay

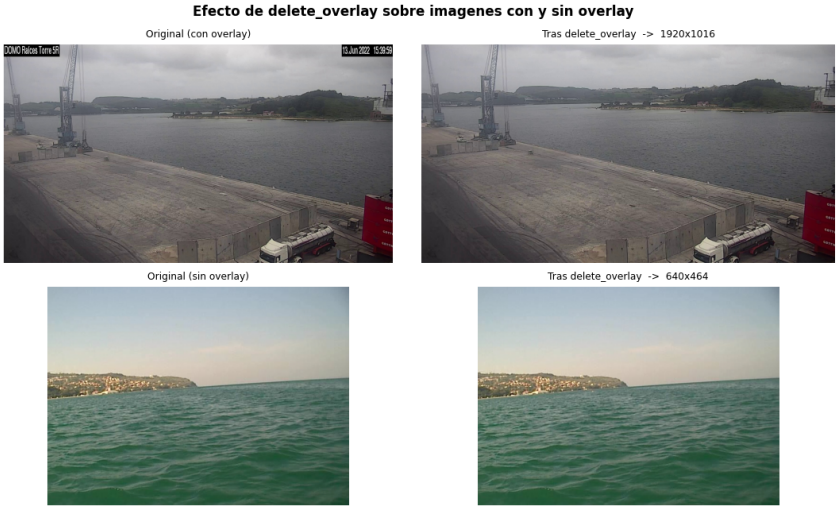
Prácticamente la totalidad de las imágenes sin *overlay* CCTV corresponden a escenas sin barco (44 No-Ship sin overlay, 0 Ship sin overlay). Esto crea una **correlación espuria** entre la presencia del overlay y la etiqueta positiva: un modelo puede aprender a detectar el overlay como síntoma de presencia de barco, sin necesidad de comprender el contenido visual. Los experimentos con overlay deben interpretarse teniéndolo en cuenta.

3. Preprocesado

3.1. Clase `PortDataset`

Se implementa una clase `Dataset` de PyTorch que carga imágenes desde rutas almacenadas en un DataFrame de pandas. Recibe dos parámetros clave:

- `label_col`: columna de etiqueta (`'ship'` o `'docked'`). Cuando se usa `'docked'`, el dataset filtra automáticamente las imágenes sin barco (`ship == 1`), ya que la tarea de atracado sólo tiene sentido sobre escenas con barco presente.
- `remove_overlay`: si es `True`, aplica la función `delete_overlay()` antes de cualquier transformación. Esta función elimina las bandas superior e inferior del overlay CCTV detectando filas de píxeles oscuros en los bordes laterales.



3.2. Partición del dataset

Se realiza un split estratificado 70 / 15 / 15 (train / val / test) usando una columna auxiliar `strat` $\in \{0, 1, 2\}$ que codifica simultáneamente el estado de Ship y Docked, garantizando que las proporciones de las tres categorías se preservan en los tres subconjuntos.

3.3. Data augmentation

Las transformaciones se diseñan a partir del análisis exploratorio del dataset, reproduciendo la variabilidad de captura real:

- **RandomHorizontalFlip**: las cámaras pueden estar orientadas en cualquier dirección horizontal.
- **ColorJitter** (brightness = 0,23, contrast = 0,19, saturation = 0,2, hue = 0,03): factores derivados de la desviación relativa (σ/μ) de brillo y contraste medida en el dataset, cubriendo la transición entre condiciones nocturnas y diurnas.
- **RandomResizedCrop(224, scale=(0.8, 1.0))**: simula la variación de escala por distancia del barco a la cámara.
- **GaussianBlur** ($\sigma_{\text{máx}} = 1{,}0$): simula niebla ligera y pérdida de nitidez a larga distancia.

Se descartan `RandomVerticalFlip` y `RandomRotation` porque las cámaras CCTV portuarias son fijas y niveladas; dichas transformaciones generarían imágenes físicamente imposibles.

4. Entrenamiento y evaluación

4.1. Arquitecturas

BaseCNN es una red convolucional de tres bloques (Conv → BN → ReLU → MaxPool) seguidos de un clasificador con una única neurona de salida.

ResNet18 preentrenado utiliza los pesos de ImageNet. Se congela el *backbone* y se sustituye la cabeza de clasificación por una neurona binaria, permitiendo el ajuste fino del modelo completo.

Ambas arquitecturas usan `BCEWithLogitsLoss` (con `pos_weight` calculado sobre el train set), optimizador AdamW, scheduler *ReduceLROnPlateau* y *early stopping* con paciencia de 10 épocas sobre el AUC de validación.

4.2. Métricas de evaluación

Dado el desbalance en Ship/No-Ship, no basta con la *accuracy*: un clasificador trivial que prediga siempre "Ship" alcanzaría ya un 62,6 %. Se utilizan por ello varias métricas complementarias, calculadas sobre el conjunto de test tras cargar el mejor *checkpoint*:

- **Accuracy**: proporción de predicciones correctas. Fiable en Docked/Undocked (clases balanceadas).
- **Precision** ($TP/(TP+FP)$) y **Recall** ($TP/(TP+FN)$). Recall es la métrica prioritaria en este dominio: no detectar un barco (FN) tiene mayor impacto operativo que una falsa alarma.
- **F1-score**: media armónica de Precision y Recall.
- **AUC-ROC**: separabilidad entre clases independiente del umbral de decisión.

Como apoyo visual, cada experimento se acompaña de las **curvas de entrenamiento** (*loss* y métricas por época sobre train y validación, útiles para diagnosticar *overfitting*) y de la **matriz de confusión** sobre test.

5. Pipeline de inferencia

Para la defensa de la práctica se ha implementado un *pipeline* de inferencia (`menu.py`) independiente del cuaderno de entrenamiento, que permite evaluar cualquiera de los 16 modelos entrenados (8 por tarea) sobre un dataset de test nuevo.

El código se divide en módulos reutilizables: `load_dataset.py` (carga y fusión de los CSVs), `PortDataset.py` y `delete_overlay.py` (clase `Dataset` y recorte del overlay, ya descritos en la sección de preprocesado), `transforms.py` (preprocesado base), `BaseCNN.py` y `build_pretrained.py` (arquitecturas, idénticas a las del entrenamiento para que el `state_dict` cargue sin conflictos) y `evaluate_model.py` (cálculo de métricas y matriz de confusión).

El `menu.py` guía al usuario mediante tres menús encadenados: **tarea** (Ship/No-Ship o Docked/Undocked), **modelo** (A, B, C, D o todos) y **dataset** (con o sin overlay). Tras la selección, carga los pesos del `.pth` correspondiente y ejecuta la evaluación.

Para usarlo sobre el dataset de defensa basta con sustituir los ficheros de `P1-Material/` (imágenes y los dos CSVs con el mismo formato) y ejecutar `python menu.py`; ninguna ruta interna necesita modificarse.

6. Resultados y análisis: Ship / No-Ship

El conjunto de test contiene 45 imágenes (28 Ship, 17 No-Ship). Los resultados se recogen en la Tabla 2.

Cuadro 2: Resultados en test — Ship / No-Ship.

Exp.	Modelo	Aug	Dataset	Acc	F1	Recall	Precision	AUC-ROC
A	BaseCNN	No	Con overlay	0,844	0,873	0,857	0,889	0,908
A	BaseCNN	No	Sin overlay	0,822	0,857	0,857	0,857	0,851
B	BaseCNN	Sí	Con overlay	0,778	0,833	0,893	0,781	0,834
B	BaseCNN	Sí	Sin overlay	0,733	0,769	0,714	0,833	0,790
C	ResNet18	No	Con overlay	0,956	0,964	0,964	0,964	0,998
C	ResNet18	No	Sin overlay	0,956	0,964	0,964	0,964	0,990
D	ResNet18	Sí	Con overlay	0,978	0,982	0,964	1,000	1,000
D	ResNet18	Sí	Sin overlay	0,956	0,966	1,000	0,933	0,957

Limitaciones estadísticas del conjunto de test

El conjunto de test consta de 45 muestras (28 ship, 17 no-ship). Con ese tamaño, un solo barco mal clasificado desplaza el recall en ≈0,036 puntos y la precision en ≈0,04. Por eso las diferencias pequeñas entre modelos (del orden de 0,02–0,05) no son concluyentes por sí solas: pueden reflejar una única predicción distinta. En un contexto portuario donde el recall es la métrica prioritaria —es preferible una falsa alarma a dejar pasar un barco sin detectar— esta incertidumbre refuerza la necesidad de un conjunto de test mayor antes de elegir un modelo para producción.

Correlaciones espurias: el overlay como señal parásita

El dataset tiene un sesgo estructural identificado en el análisis exploratorio: prácticamente todas las imágenes sin overlay corresponden a la clase No-Ship (44 sin overlay, 0 de clase Ship). Esto introduce una **correlación espuria** entre la banda de texto CCTV y la etiqueta *ship*, independiente del contenido visual.

El efecto más visible está en el experimento D con overlay: precision = 1,0 y recall = 0,964. Estos números son prometedores, pero parte de esa precision puede deberse a que el modelo aprendió que la ausencia de overlay implica No-Ship, no a que discrimine el contenido visual real. Desde la óptica del recall, **D sin overlay** es más fiable: alcanza recall = 1,0 (ningún barco sin detectar) con precision = 0,933, sin poder apoyarse en el artefacto CCTV. Ese resultado es evidencia más sólida de generalización real.

BaseCNN vs. ResNet18: la única diferencia estadísticamente robusta

La brecha entre arquitecturas supera con claridad el margen de ruido estadístico. BaseCNN oscila entre AUC 0,79 y 0,91 en todos sus experimentos, mientras que ResNet18 se sitúa entre 0,96 y 1,00. Esta diferencia de ≈0,1–0,2 en AUC es consistente en los ocho experimentos, independientemente del augmentation y del overlay: su magnitud y coherencia permiten afirmar con confianza que el preentrenamiento en ImageNet aporta representaciones visuales que no son alcanzables entrenando desde cero con ≈205 muestras.

Efecto del *data augmentation*

El impacto del augmentation es distinto según la arquitectura y debe leerse en clave de recall.

BaseCNN: con overlay, B mejora el recall respecto a A (0,893 frente a 0,857), lo que es positivo para el objetivo del puerto, aunque pierde precision (0,781 frente a 0,889) generando más falsas alarmas. Sin overlay, B cae a recall = 0,714 —el peor de todos los experimentos— mientras la precision sube a 0,833. En BaseCNN, el augmentation sin overlay penaliza gravemente la detección de barcos.

ResNet18: con overlay, D mantiene el mismo recall que C (0,964) pero sube la precision a 1,0, sin ganar capacidad de detección real. Sin overlay, D es el único experimento que alcanza recall = 1,0: ningún barco es ignorado, con precision = 0,933 (pocas falsas alarmas). Desde la perspectiva portuaria, **D sin overlay** es la configuración más valiosa: maximiza el recall sin sacrificar excesivamente la precision.

Qué podemos afirmar y qué no

Podemos afirmar: ResNet18 preentrenado supera consistentemente a BaseCNN entrenado desde cero. La diferencia es robusta y no un artefacto del test.

No podemos afirmar: cuál es el mejor modelo dentro de la familia ResNet18 (C vs. D, con vs. sin overlay). Los experimentos C y D producen métricas parecidas y estadísticamente indistinguibles con 45 muestras de test. Seleccionar un modelo para producción requeriría un conjunto de test sustancialmente mayor y datos recogidos bajo condiciones diversas que rompan los sesgos estructurales del dataset actual.

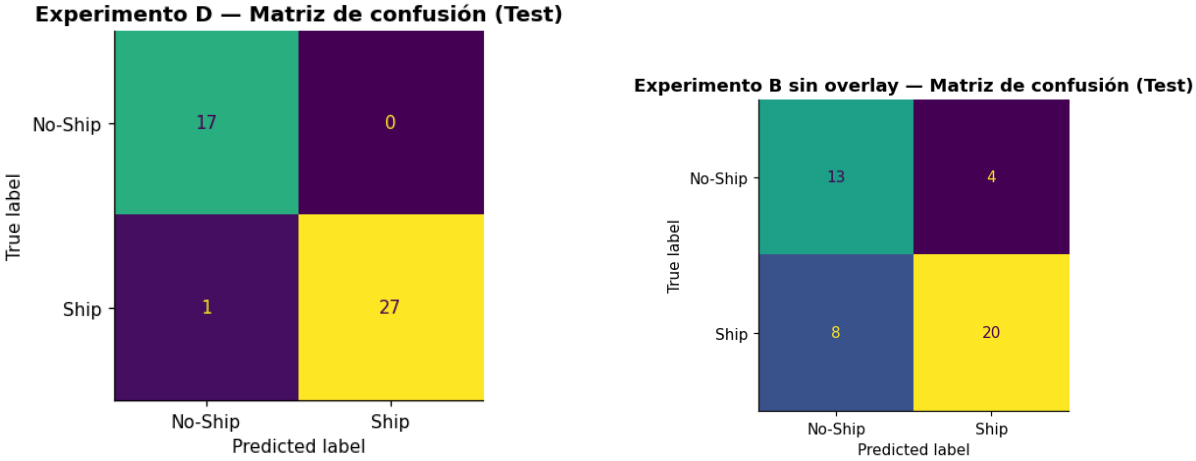


Figura 4: Mejor y Peor matriz de confusión — Ship/No-Ship

7. Resultados y análisis: Docked / Undocked

El conjunto de test contiene **28 imágenes** (14 Docked, 14 Undocked), compuesto exclusivamente por imágenes con barco presente. Las clases están perfectamente balanceadas. Los resultados se recogen en la Tabla 3.

Cuadro 3: Resultados en test — Docked / Undocked.

Exp.	Modelo	Aug	Dataset	Acc	F1	Recall	Precision	AUC-ROC
A	BaseCNN	No	Con overlay	0,536	0,629	0,786	0,524	0,658
A	BaseCNN	No	Sin overlay	0,571	0,647	0,786	0,550	0,643
B	BaseCNN	Sí	Con overlay	0,607	0,621	0,643	0,600	0,602
B	BaseCNN	Sí	Sin overlay	0,714	0,667	0,571	0,800	0,760
C	ResNet18	No	Con overlay	0,821	0,815	0,786	0,846	0,888
C	ResNet18	No	Sin overlay	0,857	0,846	0,786	0,917	0,903
D	ResNet18	Sí	Con overlay	0,750	0,774	0,857	0,706	0,913
D	ResNet18	Sí	Sin overlay	0,893	0,880	0,786	1,000	0,857

Dificultad intrínseca de la tarea

Esta tarea es más difícil que Ship/No-Ship por dos razones que el análisis exploratorio anticipó. Primero, el solapamiento en el espacio de características: la proyección UMAP mostró que los puntos Docked y Undocked se superponen casi por completo, indicando que las representaciones genéricas de ImageNet no discriminan entre estas dos clases. Segundo, la ambigüedad visual: distinguir si un barco está atracado requiere detectar amarres, la posición relativa al muelle y detalles contextuales sutiles que en muchas imágenes son difícilmente perceptibles incluso para un humano. El dataset disponible, con ≈130 ejemplos de entrenamiento balanceados entre las dos clases, es insuficiente para aprender de forma robusta estas características de alto nivel.

Limitaciones estadísticas

El subconjunto de test disponible para esta tarea consta de 28 muestras (solo imágenes con barco, ya que Docked/Undocked no aplica a las de No-Ship). Con un test tan reducido, una sola predicción cambiada altera la precision o el recall en torno a 3–5 puntos porcentuales. Por eso, diferencias entre modelos del orden de 0,03–0,05 en cualquier métrica no son significativas por sí solas: reflejan tanto el comportamiento del modelo como la partición concreta del test. Las cifras de la tabla deben leerse como orientaciones.

BaseCNN vs. ResNet18: la única diferencia robusta

Al igual que en la tarea anterior, la brecha entre arquitecturas es la única conclusión estadísticamente sólida. BaseCNN no supera AUC 0,76 en ningún escenario, mientras que ResNet18 alcanza AUC entre 0,86 y 0,91. La magnitud y consistencia de esta diferencia confirma que el preentrenamiento en ImageNet es imprescindible cuando se dispone de tan pocas muestras.

Augmentation y overlay

Augmentation: el efecto varía según la arquitectura y el compromiso recall/precision. En BaseCNN, el augmentation perjudica el recall de forma clara: A sin overlay obtiene recall = 0,786 mientras B sin overlay cae a 0,571, ganando precision (0,550 → 0,800) pero perdiendo capacidad de detectar barcos atracados. En ResNet18, el efecto es menos negativo: D obtiene el recall más alto de todos los experimentos con overlay (0,857), aunque a costa de la precision más baja en ResNet18 (0,706). La razón probable es que distinguir docked de undocked depende de señales geométricas locales (amarres, distancia al muelle, ángulo de aproximación) que transformaciones como `RandomResizedCrop` o `RandomHorizontalFlip` pueden alterar, haciendo el entrenamiento menos informativo.

Overlay: retirar el overlay mejora consistentemente la precision en ResNet18 sin alterar el recall. C sin overlay sube de precision 0,846 a 0,917 manteniendo recall = 0,786. D sin overlay alcanza precision = 1,0 (ningún falso positivo) con recall = 0,786, frente a precision 0,706 con overlay. Esto indica que el artefacto CCTV introduce ruido que genera falsos positivos (predicciones de docked incorrectas), y que eliminarlo hace al modelo más conservador y preciso en sus predicciones positivas.

Qué podemos afirmar y qué no

Podemos afirmar: ResNet18 preentrenado supera claramente a BaseCNN para esta tarea; la diferencia es suficientemente grande para ser robusta frente al ruido estadístico.

No podemos afirmar: cuál es la mejor configuración dentro de ResNet18, ni si el augmentation ayuda o perjudica de forma general. Los experimentos C y D son estadísticamente indistinguibles con 28 muestras. Los niveles de AUC alcanzados (0,86–0,91) son prometedores, pero su traducción

a capacidad de generalización real requiere validación sobre un conjunto de test significativamente mayor y con mayor diversidad de condiciones de captura.

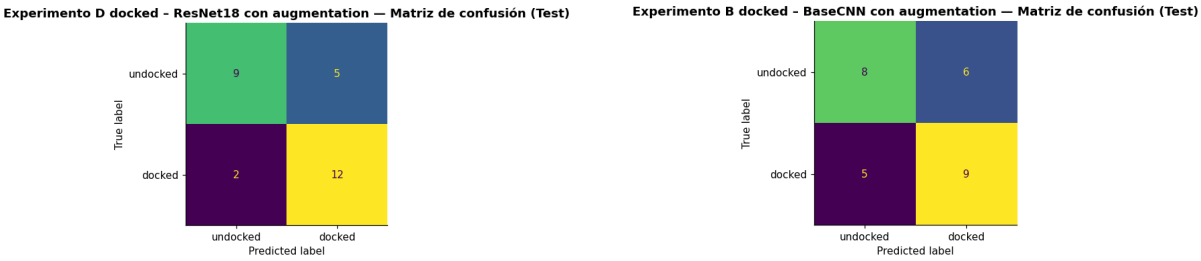


Figura 5: Mejor y Peor matriz de confusión — Ship/No-Ship

8. Conclusiones generales

- **El preentrenamiento es la variable decisiva.** La brecha entre BaseCNN y ResNet18 es la única diferencia estadísticamente robusta.
- **El análisis exploratorio guía el diseño.** El estudio previo del dataset permitió derivar los parámetros de *data augmentation* a partir de estadísticas reales (σ/μ de brillo y contraste, percentil 10 para el blur) en lugar de fijarlos arbitrariamente, y detectar el sesgo estructural del overlay. Ambas decisiones influyen directamente en la interpretación de los resultados.
- **El *data augmentation* no es universalmente beneficioso.** Su efecto depende de la arquitectura y de la tarea. En BaseCNN puede penalizar gravemente el recall sin overlay; en ResNet18 tiene efectos mixtos, mejorando unas métricas y empeorando otras. En Docked/Undocked, donde la señal discriminativa es geométrica (amarres, posición relativa al muelle), las transformaciones agresivas pueden destruir información útil.
- **El overlay introduce una correlación espuria.** Los modelos entrenados con overlay pueden apoyarse en él como de síntoma etiqueta positiva, inflando las métricas de test. Evaluar también sobre el dataset sin overlay es una forma honesta de estimar la capacidad real de generalización, especialmente en Ship/No-Ship.
- **El conjunto de test es demasiado pequeño para rankings finos.** Con 45 y 28 muestras respectivamente, diferencias de 0,02–0,05 en cualquier métrica pueden reflejar una única predicción distinta. Por eso, dentro de la familia ResNet18, los experimentos C y D son estadísticamente indistinguibles y la elección de un modelo para producción exigiría un conjunto de test mayor y más diverso.